

Agivant BOLT Guide

A Practical Workflow Guide Using GitHub Spec Kit (specify-cli)

Toolkit

GitHub Spec Kit — specify-cli v0.11.3

Repository

github.com/github/spec-kit

Methodology

Spec-Driven Development + Test-Driven Development

Table of Contents

- 1. Introduction: The Integrated Development Cycle
- 2. Tooling & Environment Setup
- 3. The 5-Step Development Workflow
- 4. How SDD and TDD Work Together
- 5. Concrete Example: Input State Gates & Totals Calculation
- 6. Best Practices for Teams.....
- 7. Repository Artifact Structure.....
- 8. Workflow Checklist.....
- 9. Conclusion
- 10. References

1. Introduction: The Integrated Development Cycle

Projects that build features directly from loose requirements force developers to make architectural and design decisions mid-coding. This leads to scope creep, architectural drift, disjointed codebases, bloated components, and poor alignment with business goals.

Combining Specification-Driven Development (SDD) with Test-Driven Development (TDD) establishes a predictable, self-documenting development cycle:

- SDD (What and Why) — defines a clear, technology-agnostic target before coding starts.
- TDD (How and Correctness) — ensures the codebase is built incrementally, remains bug-free, and adheres strictly to the target specification.

End-to-end cycle:



When requirements change after a feature ships, the cycle re-enters at Converge / Update Specs, which folds new findings back into spec.md — keeping the specification the permanent source of truth rather than letting it go stale.

Core Principle

“No implementation code is written before a failing test exists, and no task is picked up before it traces back to an approved spec.” This is treated as non-negotiable across the workflow.

2. Tooling & Environment Setup

Spec Kit is distributed as a Python CLI called `specify`, installed and run via `uv` (an ultra-fast Python package/environment manager). The steps below set up the toolchain used throughout this project.

2.1 Install uv

`uv` provides both the package manager and the isolated tool runner used to install and invoke `specify-cli`.

```

pip install uv
uv --version
  
```

2.2 Install specify-cli from the Spec Kit repository

The CLI is installed directly from the GitHub repository at a pinned release tag, ensuring a reproducible toolchain version across the team.

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git@v0.11.3

# Windows / py launcher variant
py -m uv tool install specify-cli --from git+https://github.com/github/spec-kit.git@v0.11.3
```

2.3 Verify installation

```
specify --version
specify check          # validates local environment & agent integrations
```

2.4 Initialize the project

Running `specify init` scaffolds the `.specify/` directory (templates, helper scripts) and, depending on the selected AI coding agent, a set of slash-command prompt files (e.g. `.github/prompts` for Copilot, or agent-specific equivalents).

```
# New project directory
uvx --from git+https://github.com/github/spec-kit.git specify init <PROJECT_NAME>

# OR initialize inside the current directory
specify init --here

# Skip confirmation when the directory already has files
specify init --here --force
```

During init, the CLI prompts for the AI coding agent integration in use (Claude Code, GitHub Copilot, Gemini CLI, Cursor, etc.) and generates the matching slash-command files.

3. The 5-Step Development Workflow

This is the step-by-step workflow followed to ensure design alignment, clean architecture, and robust test coverage. Steps 1–3 map directly onto the `/speckit.specify`, `/speckit.plan`, and `/speckit.tasks` commands introduced in Section 2; Steps 4–5 happen during `/speckit.implement`.

Step 1 — Feature Specification (`spec.md`)

The specification defines WHAT the feature does and WHY it is valuable. It must remain technology-agnostic — no database schemas, framework names, or library imports.

Note

The spec should be readable by both non-technical stakeholders (product managers, designers) and developers. It serves as the definitive source of truth.

Key sections of the spec:

1. Executive Summary — a concise description of the feature and business value.
2. User Scenarios & Journeys — step-by-step descriptions of the user experience.
3. Functional Requirements — atomized, testable functional points.
4. UI/UX Flow & Wireframes — layout structures, navigation paths, and user interactions.
5. Success Criteria — technology-agnostic, measurable metrics (e.g. page load speeds, maximum transaction durations).
6. Edge Cases — network failures, empty state screens, boundary constraints.

Step 2 — Implementation Plan (plan.md)

The implementation plan bridges the gap between functional requirements and technical execution. It defines HOW the feature will be built.

Key sections of the plan:

1. Technical Context — language versions, libraries, frameworks, and core dependencies.
2. Architectural Decisions — project folder layout, state management architecture, API layer separation, storage choices.
3. Constitution Check — verifying the proposed implementation aligns with the project's core architectural guidelines and governance constraints (separation of concerns, security gates, modular readability).
4. Verification & Test Plan — an outline of unit tests, integration tests, and manual verification steps.
5. Complexity Tracking — identifying potential bottlenecks or high-complexity components to monitor during development.

Step 3 — Actionable Task List (tasks.md)

The task list breaks down the implementation plan into sequential, dependency-ordered, and verifiable steps. Each task represents a small unit of work that can be implemented and tested independently.

Tasks

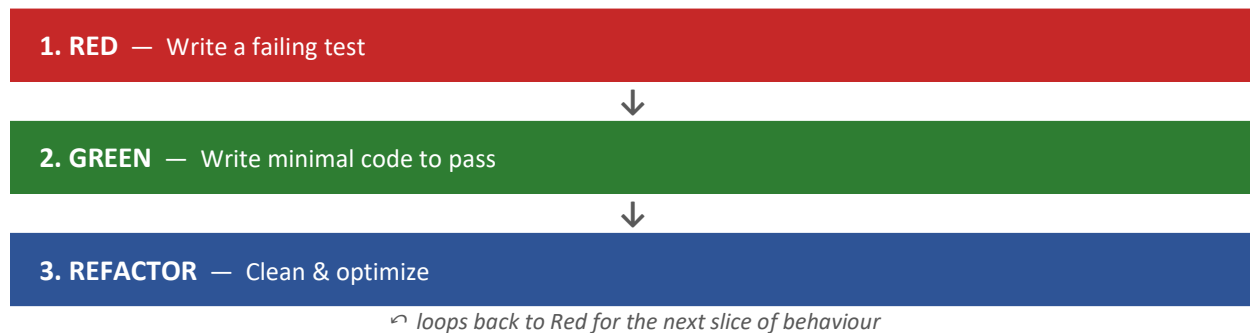
- [] Task 1: Setup data-models and utility functions
- [] Task 2: Build service layer with API handlers
- [] Task 3: Build individual components with associated unit tests
- [] Task 4: Integrate state management flow
- [] Task 5: Verify build budgets and final bundle generation

Sequence of Execution

Once the specification, plan, and task list are fully written and reviewed by the team, test cases are written for each task before any implementation code exists. Those test cases are run to verify their outcome — confirming they fail for the right reason against the current (missing) implementation. Only after that verification does code-writing begin: implementation code is written specifically to make the previously-written test cases pass. Code is therefore never written first — it is always written in response to a test case that has already been written and verified.

Step 4 — Test-First Implementation (TDD)

With the tasks defined, implementation is driven by a strict TDD micro-cycle: Red → Green → Refactor.



1. Red — write a test case that describes a functional requirement from spec.md before writing the code itself. Run the test runner and verify that it fails.
2. Green — write the minimum necessary production code to make the test compile and pass.
3. Refactor — clean up the implementation, remove duplication, improve naming, optimize performance. Run tests again to ensure they remain green.

Step 5 — Verification & Budget Compliance

The final step is to verify the implementation against the technical and performance budgets defined in the plan:

- Run the full test suite to confirm zero regressions.
- Verify production bundles compile correctly.
- Ensure component styles and chunk sizes comply with configured budget limits.

4. How SDD and TDD Work Together

SDD and TDD operate at two different altitudes of the same idea: define expected behaviour clearly, then verify it. SDD defines behaviour at the feature and product level; TDD defines and verifies it at the code level. Used together, the specification becomes the source that tests are derived from, and passing tests become the proof that the specification has been met.

Combined Workflow

Step	Action
1. Specify	Write the feature specification (what, how it behaves, data contracts, edge cases).
2. Translate to Tests	Convert each spec requirement and edge case into one or more test cases before implementation.
3. Build (Red → Green → Refactor)	Implement the code so each test passes, following the TDD cycle.
4. Verify Against Spec	Run the full suite and manually confirm the working feature matches the original specification.
5. Keep in Sync	If behaviour needs to change later, update the spec first, then the tests, then the code — in that order.

Rule of Thumb

A good rule of thumb: if a reviewer can't trace a test back to a line in the spec, either the test is unnecessary or the spec is incomplete.

5. Concrete Example: Input State Gates & Totals Calculation

A generic example of how this 5-step workflow maps from requirement, to test, to production code.

4.1 Specification (Step 1)

In the feature spec (spec.md), a requirement states:

Requirement

“The configuration wizard must display a list of periods. The user can add, duplicate, or delete periods. The screen must dynamically calculate totals for period months and amounts. The navigation button to the next tab remains disabled until all periods contain valid values.”

4.2 Test-First Implementation (Step 4 — RED)

Before writing the wizard component's logic, unit tests are written mapping directly to these requirements:

```
describe('Wizard Component - Configuration Flow', () => {
  let component: WizardComponent;
  let fixture: ComponentFixture<WizardComponent>;

  beforeEach(() => {
    // Component test setup...
  });

  describe('Dynamic Calculation Logic', () => {
    it('should calculate totals correctly when periods are updated', () => {
      component.periods = [
        { months: '12', amount: 50000 },
        { months: '24', amount: 100000 }
      ];

      let emittedMonths = 0;
      let emittedAmount = 0;
      component.totalsChanged.subscribe((event) => {
        emittedMonths = event.months;
        emittedAmount = event.amount;
      });

      component.calculateTotals();

      // Verify the emitted totals match expectations
      expect(emittedMonths).toBe(36);
      expect(emittedAmount).toBe(150000);
    });
  });

  describe('State Navigation Gates', () => {
    it('should disable navigation if any period value is missing', () => {
      component.periods = [{ months: '', amount: null }]; // Invalid period
      expect(component.canNavigateToNextTab()).toBeFalse();

      component.periods = [{ months: '12', amount: 50000 }]; // Valid period
      expect(component.canNavigateToNextTab()).toBeTrue();
    });
  });
});
```


4.3 Production Code (Step 4 — GREEN)

Next, the minimal implementation code is written to satisfy the tests:

```
export class WizardComponent {
  periods: Period[] = [];
  totalsChanged = new EventEmitter<{months: number, amount: number}>();

  calculateTotals(): void {
    let totalMonths = 0;
    let totalAmount = 0;

    for (const period of this.periods) {
      if (period.months) {
        totalMonths += parseInt(period.months, 10) || 0;
      }
      if (period.amount) {
        totalAmount += period.amount;
      }
    }

    this.totalsChanged.emit({ months: totalMonths, amount: totalAmount });
  }

  canNavigateToNextTab(): boolean {
    if (this.periods.length === 0) return false;
    // Check if any period has empty or missing values
    const hasInvalidPeriod = this.periods.some(p => !p.months || p.amount === null);
    return !hasInvalidPeriod;
  }
}
```

6. Best Practices for Teams

To successfully scale this workflow across a development team:

1. Enforce the “Specification First” rule — do not start writing code or plan tasks until the feature specification (spec.md) has been reviewed and signed off by the team.
2. Keep spec and code in sync — as design changes occur during implementation, immediately update the spec and plan first, rather than letting code and documentation diverge.
3. Automated CI/CD verification — run tests and verify build compilation on every pull request to ensure no budget violations or failing tests reach the main branch.

7. Repository Artifact Structure

Each feature branch accumulates its own set of Markdown artifacts under a numbered feature folder, keeping specs versioned alongside code rather than in an external tool.

```
.specify/                # Spec Kit templates & helper scripts
memory/
  constitution.md         # Project-wide non-negotiable principles
specs/
  001-feature-name/
    spec.md               # WHAT & WHY (from /speckit.specify)
    plan.md               # HOW — stack & architecture (from /speckit.plan)
    tasks.md              # Ordered task breakdown (from /speckit.tasks)
    checklist.md          # Quality checklist (from /speckit.checklist)
  002-next-feature/
  ...
```

8. Workflow Checklist

Phase	Developer Action → Deliverable
1. Specify	Write user scenarios, requirements, and criteria. → spec.md
2. Plan	Outline stack, architecture, and test plan. → plan.md
3. Tasking	Break down work into atomic checkboxes. → tasks.md
4. TDD (Red)	Write failing test cases matching spec. → Unit tests
5. TDD (Green)	Write code to make test cases pass. → Production file updates
6. Refactor	Optimize architecture and styling. → Clean code
7. Verify	Run build script and test suite. → Successful compile

9. Conclusion

SDD and TDD work best when practiced as a single, continuous pipeline — not as separate methodologies bolted together. The spec defines *what* to build, the tests define *when it's done*, and the code satisfies both. Because every test traces back to a spec requirement, and every line of code exists to pass a test, the three artifacts stay aligned by design — not by discipline alone.

Teams that adopt this workflow don't just ship working software. They ship software with a built-in audit trail: every feature has a spec that explains *why*, a test that proves *what*, and code that delivers *how*. When requirements change, the same sequence runs again — spec first, tests second, code third — keeping the entire system honest.

10. References

- GitHub Spec Kit repository: github.com/github/spec-kit
- Spec-Driven Development methodology deep dive: [spec-kit/spec-driven.md](#)
- Spec Kit official documentation & quick start guide